

講義 17：運算與流程控制（授課順序：接在講義 2 之後）

對應練習：ex17 | 答案程式：ZR_TR17_CONTROL_FLOW

本講重點

- 算術運算子： $+$ $-$ $*$ $/$ 、整數商 **DIV**、餘數 **MOD**
- 條件分支：**IF** / **ELSEIF** / **ELSE**、**CASE** / **WHEN**
- 比較運算子與邏輯運算：**AND** / **OR** / **NOT**、**IS INITIAL**、**BETWEEN**
- 迴圈：**DO n TIMES**、**DO**（無上限）、**WHILE**
- 迴圈控制三兄弟：**EXIT**、**CONTINUE**、**CHECK**

1. 算術運算

```
DATA: gv_a      TYPE i VALUE 17,  
      gv_b      TYPE i VALUE 5,  
      gv_result TYPE i,  
      gv_avg     TYPE p LENGTH 8 DECIMALS 2.  
  
gv_result = gv_a + gv_b.      " 22  
gv_result = gv_a - gv_b.      " 12  
gv_result = gv_a * gv_b.      " 85  
gv_result = gv_a DIV gv_b.     " 3    整數商（無條件捨去）  
gv_result = gv_a MOD gv_b.     " 2    餘數  
gv_avg    = gv_a / gv_b.       " 3.40  
gv_result = gv_a / gv_b.       " 3 ! 目的地是整數 → 四捨五入
```

兩個必知：

- **結果存哪裡決定精度**：**17 / 5** 存進 **i** 得 3（捨入）、存進 **p DECIMALS 2** 得 3.40。金額計算的中間變數就要用 **p**。
- **除以零會 dump**（執行期錯誤 **COMPUTE_INT_ZERODIVIDE**）：分母來自資料時，除之前必須檢查不為零——講義 19 的除錯練習就埋了這顆地雷。

舊程式會看到 **COMPUTE gv_x = ...**（跟 **=** 等價）與 **ADD 1 TO gv_x.** 等舊寫法——看得懂即可，新程式一律用 **=**。

2. IF / ELSEIF / ELSE

```
IF gv_score >= 80.  
    gv_grade = 'A'.  
ELSEIF gv_score >= 60.  
    gv_grade = 'B'.  
ELSE.
```

```
gv_grade = 'C'.  
ENDIF.
```

- 由上往下判斷，第一個成立的分支執行完就跳出——所以條件要從嚴到寬排（先 ≥ 80 再 ≥ 60 ）。
- **ELSEIF** / **ELSE** 都可省略；**ENDIF** 不可省略。

2.1 比較與邏輯運算子

符號寫法	舊字母寫法（看得懂即可）	意義
=	EQ	等於
<>	NE	不等於
> / >=	GT / GE	大於 / 大於等於
< / <=	LT / LE	小於 / 小於等於

組合條件用 **AND** / **OR** / **NOT**，必要時加括號讓優先順序一目了然：

```
IF ( gv_score >= 60 AND gv_score < 80 ) OR gv_makeup = 'X'.
```

常用判斷式（不是比較符號、是**述語**）：

IF gv_name IS INITIAL.	" 是不是初始值（空白/0）
IF gv_score BETWEEN 60 AND 79.	" 區間（含兩端）
IF gv_grade CA 'AB'.	" Contains Any：含 A 或 B 任一字元
IF gv_carrid IN s_carrid.	" range 條件（講義 7）

ABAP 沒有布林型別：旗標欄位慣例用 **c LENGTH 1**，'X' = 真、空白 = 假（checkbox 也是這樣，講義 7）。判斷寫 **IF p_flag = 'X'.**，別寫 **IF p_flag.**（語法錯誤）。

3. CASE / WHEN：離散值分支

同一個變數對多個「特定值」分支時，CASE 比一串 IF/ELSEIF 清楚：

```
CASE gv_grade.  
  WHEN 'A'.  
    WRITE / '優等'.  
  WHEN 'B' OR 'C'.           " 多值共用一個分支  
    WRITE / '普通'.  
  WHEN OTHERS.              " 其餘全部（建議永遠要寫）  
    WRITE / '資料異常'.  
ENDCASE.
```

- CASE 比的是**完全相等**；範圍條件（ ≥ 60 ）只能用 IF。
- **WHEN OTHERS** 接住預期外的值——不寫的話，比不中就默默什麼都不做，容易吞掉資料異常。

4. 迴圈：DO 與 WHILE

* 固定次數：DO n TIMES，圈數在 sy-index (1 起算)

```
DO 5 TIMES.
```

```
  WRITE: / '第', sy-index, '圈'.
```

```
ENDDO.
```

* 不定次數：DO 沒有上限，必須自己 EXIT，否則無窮迴圈！

```
DATA gv_sum TYPE i.
```

```
DO.
```

```
  gv_sum = gv_sum + sy-index.
```

```
  IF gv_sum > 100.
```

```
    EXIT. " 跳出迴圈
```

```
  ENDIF.
```

```
ENDDO.
```

* 前測式：WHILE 條件成立才進圈

```
DATA gv_n TYPE i VALUE 1.
```

```
WHILE gv_n * gv_n <= 200. " 找出平方不超過 200 的最大整數
```

```
  gv_n = gv_n + 1.
```

```
ENDWHILE.
```

- **sy-index** 是 DO/WHILE 的圈數；內表的 **LOOP AT** 用的是 **sy-tabix** (講義 4)，兩個別搞混——巢狀時內層會蓋掉外層的值，需要就先存進自己的變數。
- 無上限 **DO** 一定要有出口；手滑寫出無窮迴圈時，程式最終以 **TIME_OUT dump** 收場。

5. EXIT / CONTINUE / CHECK

三個指令控制「這一圈 / 整個迴圈」怎麼走，LOOP、DO、WHILE 通用：

指令	效果
EXIT	跳出 整個迴圈 ，繼續執行 ENDDO/ENDWHILE 之後的程式
CONTINUE	跳過這一圈剩下的程式，直接進下一圈
CHECK 條件	條件 不成立 就等同 CONTINUE (成立才往下走) ——常放圈首當過濾器

```
DO 10 TIMES.
```

```
  CHECK sy-index MOD 2 = 0. " 奇數直接下一圈
```

```
  WRITE: / '偶數：', sy-index.
```

```
ENDDO.
```

兩個注意：

- **CHECK** 讀起來是「通過檢查才繼續」，跟 IF 的語感相反，第一次讀舊程式很容易看反——它在 ZDQM 等舊程式裡極常見。
- **EXIT** 寫在**迴圈外** (例如直接在 START-OF-SELECTION 或 FORM 裡) 意義完全不同：結束目前處理區塊，效果近似 RETURN——同一個字兩種行為，建議迴圈外要提早結束一律用 **RETURN**，語意單一。

6. 常見錯誤與陷阱

症狀	原因
平均值永遠是整數	結果存進 <code>i</code> ——中間與結果變數改用 <code>p DECIMALS</code>
執行期 dump COMPUTE_INT_ZERODIVIDE	除法前沒檢查分母為零
IF 分支順序對了條件卻進錯	條件從寬排到嚴 (<code>>= 60</code> 寫在 <code>>= 80</code> 前面)，第一個成立就出去了
CASE 對範圍值不動作	CASE 只比完全相等，範圍判斷用 IF
程式跑到 TIME_OUT	無上限 DO/WHILE 沒有出口，或出口條件永遠不成立
CHECK 邏輯寫反	CHECK 是「成立才往下」，不是「成立就跳過」

7. 課堂練習

完成 [ex17](#)：算術運算與 DIV/MOD、同一個等第判斷分別用 IF 與 CASE 寫、DO/WHILE 求和、CHECK 過濾偶數。